

Suite de pruebas y cobertura Ae6

Universidad Espíritu Santo · Diseño de Software UCOM0310
Semana 7 · Actividad individual · 29 de septiembre de 2026

Estudiante: Jordy Alessandro Fajardo Figueroa
Carrera: Ingeniería en Ciencias de la Computación
Materia: Diseño de Software

1 Objetivo y resultado

Proteger las reglas de cancelación, cálculo de descuentos y confirmación del proyecto base mediante pruebas unitarias reproducibles. La entrega ejecutó 16 pruebas sin fallos y alcanzó 100 % de líneas y ramas en ReservaService. La mejora se documentó con evidencia previa y final, commits incrementales y un Pull Request hacia main.

2 Reglas y matriz de casos

Se permite cancelar desde dos horas de anticipación. NORMAL conserva el total, VIP descuenta 15 % y ESTUDIANTE 10 %; una base negativa se rechaza. Confirmar exige una reserva y disponibilidad: en caso exitoso cambia el estado, guarda y notifica. Los casos cubren entradas habituales, fronteras y rechazos.

ID	Método y entrada	Esperado	Tipo
01	Cancelar 5 horas	true	Normal
02	Cancelar 2 horas	true	Límite válido
03	Cancelar 1 hora	false	Límite inválido
04	NORMAL, 100	100	Normal
05	VIP, 100	85	Alternativo
06	ESTUDIANTE, 100	90	Alternativo
07	VIP, 0	0	Límite
08	NORMAL, -1	IllegalArgumentException	Inválido
09	R-001 disponible	CONFIRMADA, guardar y notificar	Normal
10	R-002 no disponible	IllegalStateException, PENDIENTE	Alternativo
11	Reserva null	IllegalArgumentException, sin consultas	Inválido
12	Cancelar 0 horas	false	Límite
13	Cancelar -1 hora	false	Inválido
14	vip, 100	85	Alternativo
15	estudiante, 100	90	Alternativo

3 Implementación JUnit y AAA

ReservaServiceTest usa cinco invocaciones parametrizadas para cancelación, seis para descuentos y una prueba de total negativo. ConfirmacionTest añade tres pruebas del flujo de confirmación. Se conserva entornoJUnitFunciona como prueba de arranque; no se cuenta como protección de una regla de negocio. Los quince casos de la matriz corresponden a estas quince invocaciones.

Arrange prepara las entradas, una reserva real y los colaboradores controlados; Act ejecuta el método bajo prueba; Assert compara el resultado, el estado o la excepción y las interacciones pertinentes. En los parámetros, Arrange reside en CsvSource. Los cálculos usan una tolerancia de 0,000001 para comparar double. Las excepciones comprueban tipo y mensaje.

Los casos de 2 y 1 horas detectan errores de frontera como reemplazar mayor o igual por mayor. Los esperados 85 y 90 protegen las tasas de descuento; la base cero se acepta y -1 se rechaza. No se modificó el código productivo para obtener pruebas verdes.

4 Stub y Mock utilizados

DisponibilidadClient actúa como Stub: when(...).thenReturn(true/false) controla el dato externo y permite probar ambos caminos de forma determinista. Aunque Mockito crea el doble, su papel en estos casos es proporcionar la disponibilidad, sin depender de una consulta real.

ReservaRepository y Notificador actúan como Mock: verify exige guardar y enviar confirmación una vez con la misma instancia de Reserva. Son efectos observables del flujo exitoso. Ante indisponibilidad, verifyNoInteractions exige que no se persista ni notifique una reserva rechazada; el estado permanece PENDIENTE. Ante null, tampoco se consulta disponibilidad. No se exige un orden interno adicional que el contrato no especifica.

5 Ejecución desde cero

Comando: **mvn -B clean test**. Entorno: Java 21.0.12.1 y Maven 3.9.16. Evidencia final: docs/evidencias/03_ejecucion_final.txt. Resultado: **BUILD SUCCESS; Tests run: 16, Failures: 0, Errors: 0, Skipped: 0**. Finalizó el 29 de septiembre de 2026 a las 22:09:52, hora de Ecuador.

El equipo tenía Java 26 por defecto, incompatible con la instrumentación de JaCoCo 0.8.12. Se seleccionó el Java 21 ya instalado en la extensión Java de VS Code mediante JAVA_HOME. Para reproducir en otro equipo basta un JDK 21 propio y verificar mvn -version. Mockito emitió advertencias de carga dinámica del agente; las pruebas y el reporte finalizaron correctamente.

6 Cobertura JaCoCo e interpretación

La medición previa tenía 15 pruebas y dejó sin ejecutar el rechazo de reserva nula. El HTML y los contadores mostraron una línea y una rama pendientes en confirmar. Se añadió CP-11, que exige `IllegalArgumentException` y cero interacciones con los tres colaboradores. La evidencia de la secuencia está en los logs y CSV previos y finales.

ReservaService	Antes	Después
Líneas	20/21 = 95,24 %	21/21 = 100 %
Ramas	11/12 = 91,67 %	12/12 = 100 %
Instrucciones	69/74 = 93,24 %	74/74 = 100 %
Métodos	4/4 = 100 %	4/4 = 100 %

En la medición previa, confirmar era el método con menor cobertura del servicio por el camino `reserva == null`. Después de la mejora, sus cuatro métodos tienen líneas y ramas completas. El reporte HTML se regenera con `mvn clean test` y se abre en `target/site/jacoco/index.html`; target permanece fuera de Git.

El proyecto completo cubre 34/39 líneas (87,18 %) y 15/18 ramas (83,33 %). Reserva cubre 9/14 líneas y 3/6 ramas: siguen pendientes los identificadores inválidos, el tipo nulo, `getId`, `getTipo` y `cancelar`. `EstadoReserva` cubre 4/4 líneas y no tiene ramas medidas. No debe confundirse la cobertura completa del servicio con la del dominio.

Cobertura alta indica ejecución, no validez de todas las salidas. Un descuento VIP incorrecto podría atravesar las mismas ramas; la aserción que exige 85 sobre 100 es la que detecta el error. Además, las ramas completas no prueban todas las combinaciones de entradas ni fallos de infraestructura.

7 Flujo Git y commits

Línea base conocida: main en 10a1565. Rama específica: ae6/suite-pruebas. La matriz ya estaba registrada antes de ampliar las pruebas. Se publicaron commits separados para reglas, confirmación, mejora derivada de cobertura y documentación.

Commit	Propósito
a300a26	Diseñar matriz de casos Ae6
1e78924	Cubrir límites de cancelación y descuentos
ce9594d	Verificar confirmación con Stub y Mock
a7ca128	Cerrar rama de reserva nula detectada con JaCoCo
54bbce7	Interpretar JaCoCo y completar autorrevisión

Anexo: docs/evidencias/05_historial_git.txt. El historial posterior incorpora la publicación del reporte técnico. Comandos de consulta: `git log --oneline --decorate` y `git diff main...ae6/suite-pruebas`.

8 Pull Request y autorrevisión

Pull Request 1: <https://github.com/jordyfajardo-17/UEES-Ae6-Semana7/pull/1>

Título: test: completar suite, cobertura y evidencia de Ae6. Base: main; compare: ae6/suite-pruebas. Fue fusionado con main por el estudiante; conserva los cambios y commits para revisión. Incluye objetivo, cambios, reproducción, cobertura, limitaciones y declaración de IA.

La revisión del diff comprobó los nombres expresivos, límites, resultados de negocio, mensajes de excepciones y efectos de confirmación. Se verificó que producción no cambió, que target está ignorado y que no se añadieron credenciales. Los logs y CSV incluidos son anexos de evidencia seleccionados; no se versionó el directorio de compilación. La lista está en docs/04_AUTORREVISION.md.

9 Conclusiones y trabajo futuro

La suite protege las reglas mínimas de Ae6 y permite repetir la ejecución sin servicios externos. La matriz identifica el riesgo de cada caso y la evidencia antes/después demuestra una mejora guiada por JaCoCo. Git y el PR permiten reconstruir la evolución y revisar el cambio frente a una línea base conocida.

La entrega tiene alcance unitario. No valida implementaciones reales de persistencia, disponibilidad o notificaciones, ni concurrencia o atomicidad. Si guardar falla, la reserva ya fue confirmada en memoria: hace falta definir la política de recuperación antes de exigir un comportamiento mediante pruebas. Tampoco se especifican NaN, infinito, redondeo o tipos desconocidos para los importes.

Como trabajo futuro, se propone acordar esos contratos, cubrir las validaciones y transiciones del dominio y añadir pruebas de integración de los adaptadores. Una evaluación adicional mediante mutaciones ayudaría a comprobar que las aserciones detectan cambios en fronteras y tasas de descuento.

10 Declaración de uso de IA

Se utilizó OpenAI Codex para diseñar y ampliar casos, implementar JUnit y Mockito, ejecutar Maven, interpretar JaCoCo, preparar documentos y registrar/publicar Git y el Pull Request. La ejecución y las métricas declaradas se respaldan con logs y CSV del proyecto. La autorrevisión fue asistida por IA y no representa una aprobación independiente; corresponde al estudiante revisar y comprender la entrega antes de presentarla.

Evidencia entregada

docs/01_MATRIZ_CASOS_PLANTILLA.md: matriz completa y trazabilidad.

docs/02_ANALISIS_COBERTURA_PLANTILLA.md: interpretación y límites.

docs/03_PULL_REQUEST_PLANTILLA.md: descripción de PR.

docs/04_AUTORREVISION.md: revisión técnica.

docs/evidencias/: logs de ejecución, CSV de cobertura y Git.

README.md: requisitos y reproducción.